

数据库设计

1.数据库设计概述

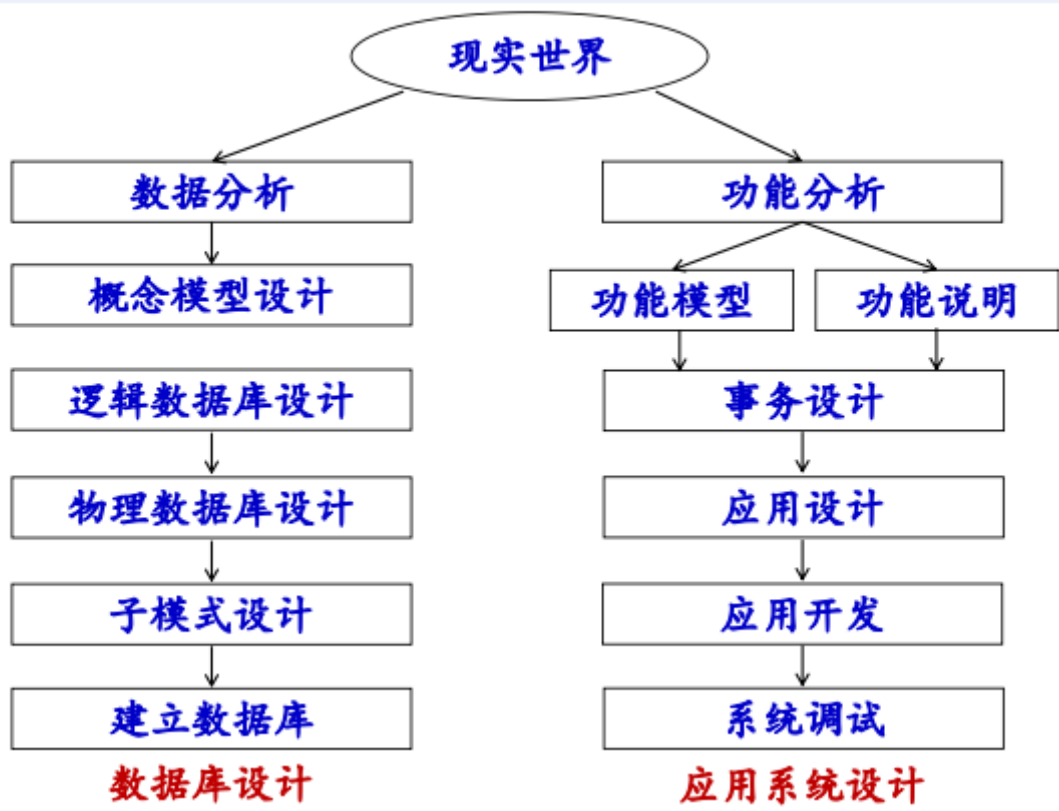
数据库设计是指对于一个给定的应用环境，设计优化的数据库逻辑模式和物理结构，并据此建立数据库及其应用系统，使之能够有效存储和关联数据，满足各种用户的应用需求，包括信息管理要求和数据操作要求。

数据库设计的目标：为用户和各种应用系统提供一个信息基础设施和高效率的运行环境。

1.1 数据库设计特点

- 1. 数据库建设的基本规律：
 - 三分技术、七分管理、十二分基础数据
 - 管理：数据库项目管理、企业业务管理
 - 基础数据：数据的收集、整理、组织和更新
- 2. 结构（数据）设计和行为（处理）设计相结合：
 - 两者相结合
 - 结构和行为分离的设计：
 - 传统软件工程：重行为设计
 - 早期数据库设计：重结构设计

图7.1 结构和行为分离的设计



1.2 数据库设计方法

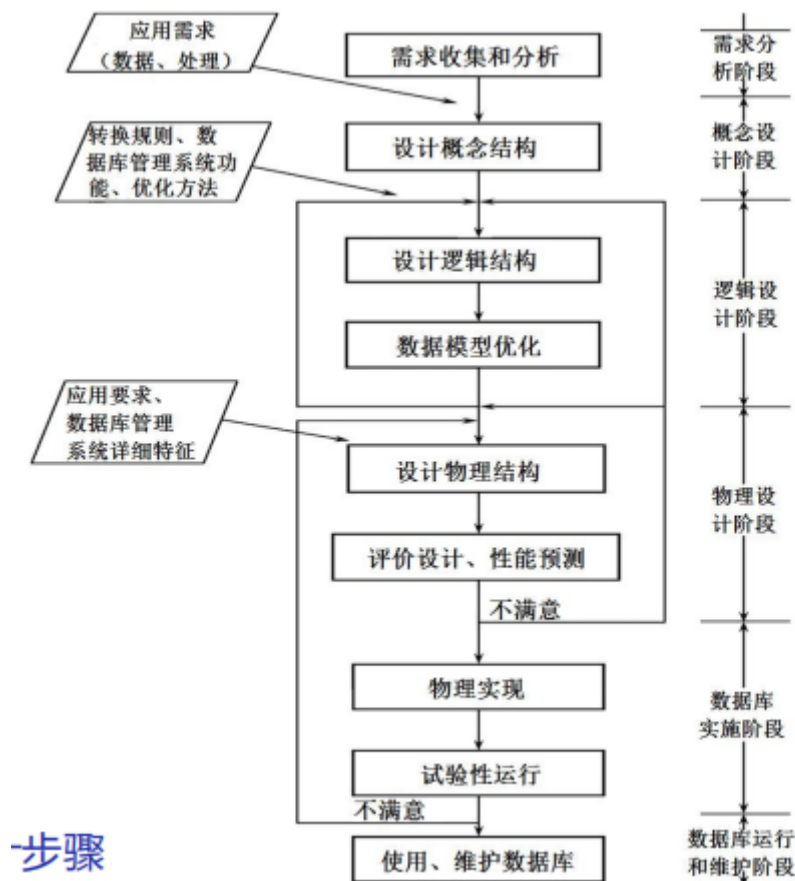
- 手工试凑法（手工和经验相结合）：缺乏科学理论和工程方法和支持，质量难以保证；维护代价高
- 规范设计法（基本思想：过程迭代和逐步求精）：
 - 新奥尔良方法
 - 基于ER模型的数据库设计方法
 - 3NF设计方法
 - 面向对象的设计方法
 - UML方法

1.3 基本步骤

分为6个阶段：

- 需求分析
- 概念结构设计
- 逻辑结构设计
- 物理结构设计
- 数据库实施
- 数据库运行和维护

需求分析和概念设计独立于任何DBMS，逻辑设计和物理设计与选用的DBMS密切相关。



参与数据库设计的人员：

➤ **系统分析人员 和 数据库设计人员**

- 自始至终参与数据库设计，其水平决定了数据库系统的质量

➤ **数据库管理员 和 用户代表**

- 主要参加需求分析与数据库的运行和维护

➤ **应用开发人员**

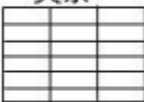
- 包括程序员和操作员
- 在实施阶段参与进来，分别负责编制程序和准备软硬件环境

各个步骤的具体操作：

1. 需求分析阶段 ➤ 是否充分与准确，决定了构建数据库的速度和质量	4. 物理结构设计阶段 ➤ 为逻辑数据结构选取一个最适合应用环境的物理结构，包括存储结构和存取方法
2. 概念结构设计阶段 ➤ 通过对用户需求进行综合、归纳与抽象，形成一个独立于具体数据库管理系统的概念模型	5. 数据库实施阶段 ➤ 根据逻辑设计和物理设计的结果构建数据库 ➤ 编写与调试应用程序 ➤ 组织数据入库并进行试运行
3. 逻辑结构设计阶段 ➤ 将概念结构转换为某个数据库管理系统所支持的数据模型，并对其进行优化	6. 数据库运行和维护阶段 ➤ 经过试运行后即可投入正式运行 ➤ 在运行过程中必须不断对其进行评估、调整与修改

- 设计一个完善的数据库应用系统是上述6个阶段的不断重复

数据库设计各个阶段的数据设计描述：

设计阶段	设计描述
需求分析	数字字典、全系统中数据项、数据结构、数据流、数据存储的描述
概念结构设计	概念模型 (E-R 图)  数据字典
逻辑结构设计	某种数据模型 关系  非关系 
物理结构设计	存储安排 存取方法选择 存取路径建立 
数据库实施	创建数据库模式 装入数据 数据库试运行 
数据库运行和维护	性能监测、转储/恢复、数据库重组和重构

1.4 各级模式

❑ 数据库设计不同阶段形成的数据库各级模式

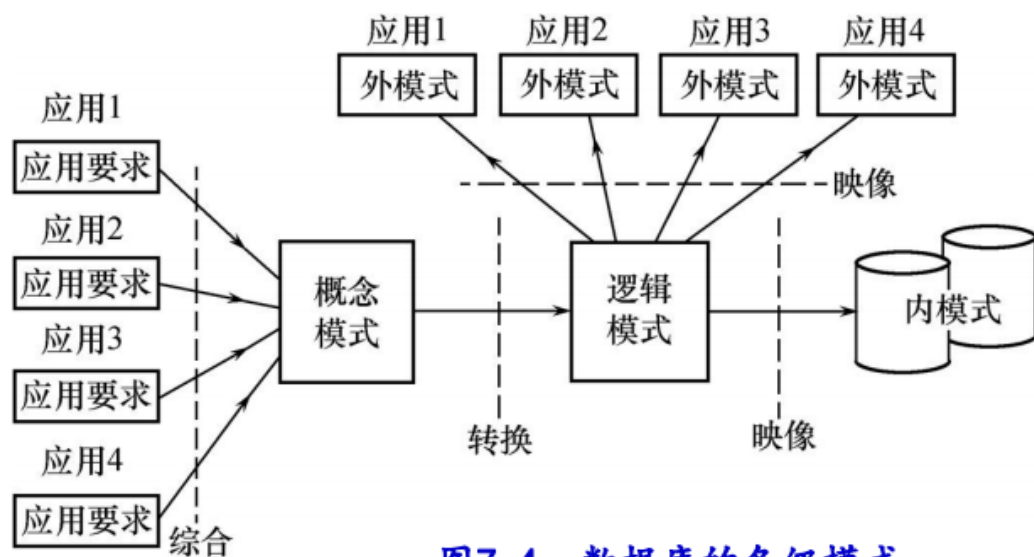


图7.4 数据库的各级模式

- 需求分析阶段：综合各个用户的应用要求
- 概念设计阶段：形成独立于机器特点、独立于各个数据库管理系统产品概念模式（ER图）
- 逻辑设计阶段：
 - 首先将ER图转换成具体的数据库产品支持的数据模型，如关系模型，形成数据库逻辑模式
 - 然后根据用户处理的要求、安全性的考虑，在基本表的基础上再建立必要的视图，形成数据的外模式
- 物理设计阶段：根据数据库管理系统特点和处理的需要，进行物理存储安排，建立索引，形成数据库内模式

2.需求分析

2.1 需求分析的任务

需求分析就是分析用户的要求。

- 详细调查现实世界要处理的对象：调查的重点是数据和处理：信息要求、处理要求、安全性和完整性要求
- 充分了解原系统工作概况
- 明确用户的各种需求：难点：用户提出的需求往往不断变化；设计者可能误解需求——>需要双方深入交流
- 在此基础上确定新系统的功能
- 新系统必须考虑可能的改变

2.2 需求分析的方法

- 调查清除用户的实际需求并进行初步分析
- 与用户达成共识
- 分析与表达这些需求

2.2.1 调查用户需求的步骤

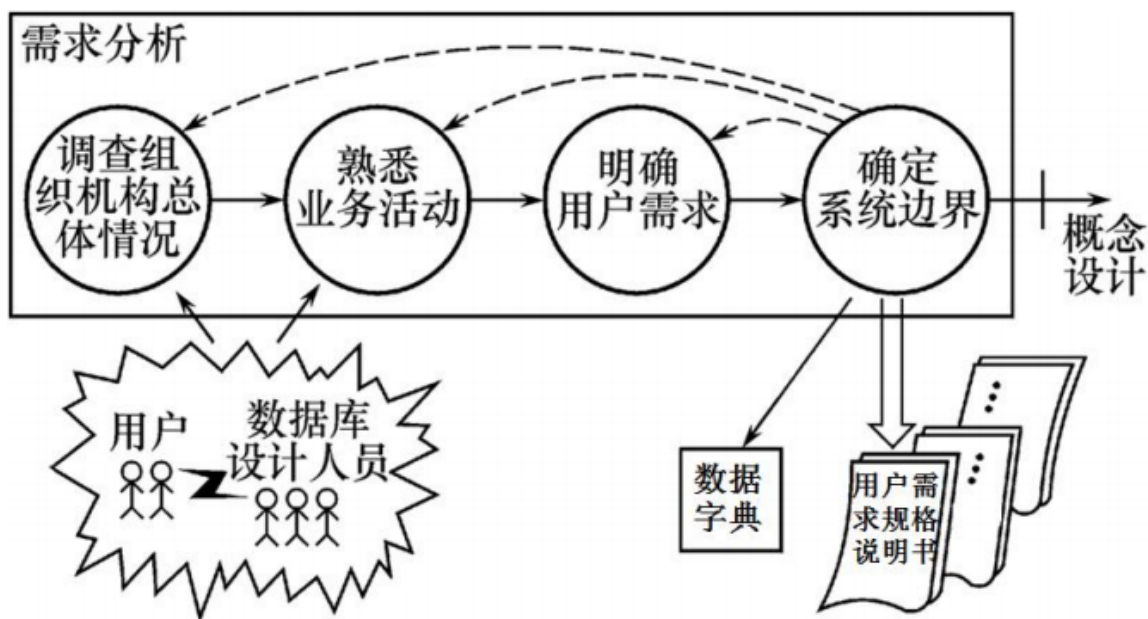
- 调查组织机构情况
- 调查各部门业务活动情况
- 在熟悉业务活动的基础上，协助用户明确对新系统的各种要求
- 确定新系统的边界

2.2.2 常用调查方法

- 跟班作业
- 开调查会
- 请专人介绍
- 询问
- 设计调查表请用户填写
- 查阅记录

进一步分析和表达用户需求：结构化分析方法（自顶向下、逐层分解）

图7.5 需求分析过程



2.3 数据字典

2.3.1 定义

数据字典包括：数据项、数据结构、数据流、数据存储、处理过程。

数据项是数据的最小组成单位。若干数据项组成一个数据结构。

数据字典通过对数据项和数据结构的定义来描述数据流、数据存储的逻辑内容。

2.3.2 数据项

数据项描述={数据项名，数据项含义说明，别名，数据类型，长度，取值范围，取值含义，与其他数据项的逻辑关系，数据项之间的联系}

2.3.3 数据结构

数据结构反映了数据之间的组合关系。

数据结构的描述={数据结构名, 含义说明, 组成: {数据项或数据结构}}

2.3.4 数据流

数据流是数据结构在系统内传输的路径。

数据流描述={数据流名, 说明, 数据流来源, 数据流去向, 组成: {数据结构}, 平均流量, 高峰期流量}

2.3.5 数据存储

数据存储是数据结构停留或保存的地方, 也是数据流来源和去向之一。

数据存储描述={数据存储名, 说明, 编号, 输入的数据流, 输出的数据流, 组成: {数据结构}, 数据量, 存取频度, 存取方式}

存取方式: 批处理/联机处理; 检索/更新; 顺序检索/随机检

2.3.6 处理过程

处理过程的具体处理逻辑一般用判定表或判定树描述。

处理过程描述={处理过程名, 说明, 输入: {数据流}, 输出: {数据流}, 处理: {简要说明}}

简要说明: 说明该处理过程的功能和处理要求。

- 处理要求是后面物理设计的输入及性能评价的标准

2.4 小结

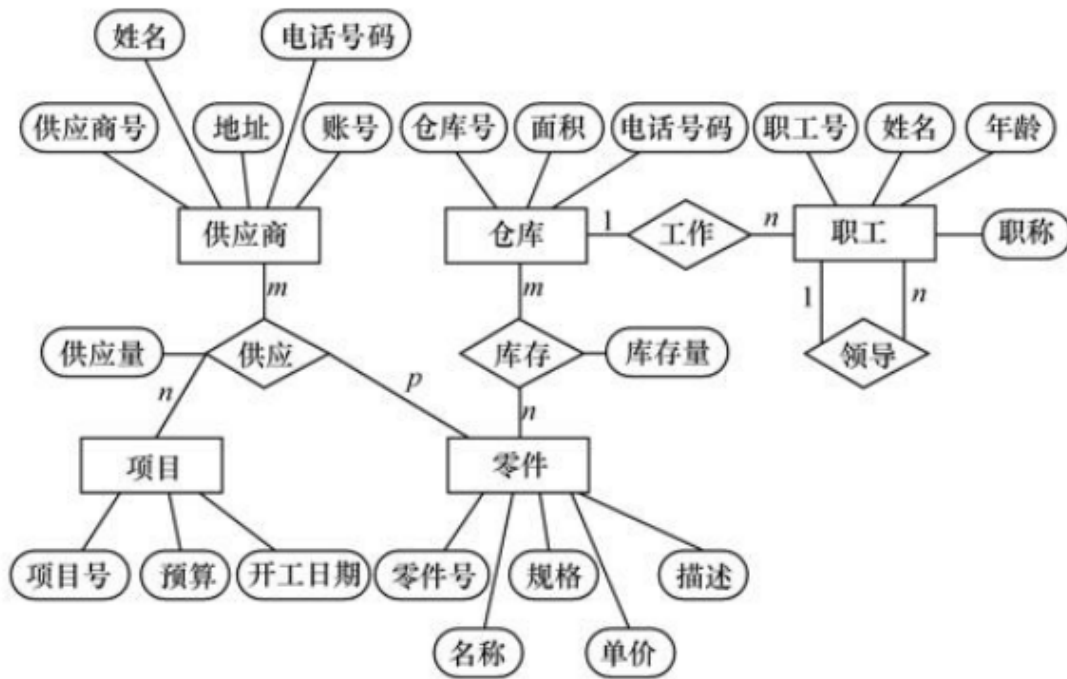
- 设计人员应充分考虑可能的扩充和改变, 使设计易于更改, 系统易于扩充
- 必须强调用户的参与

3.概念结构设计

3.1 概念模型

3.2 ER模型

E-R图实例 4



(c) 完整的实体-联系图

关于概念模型的定义以及ER模型详见：[概念模型](#)

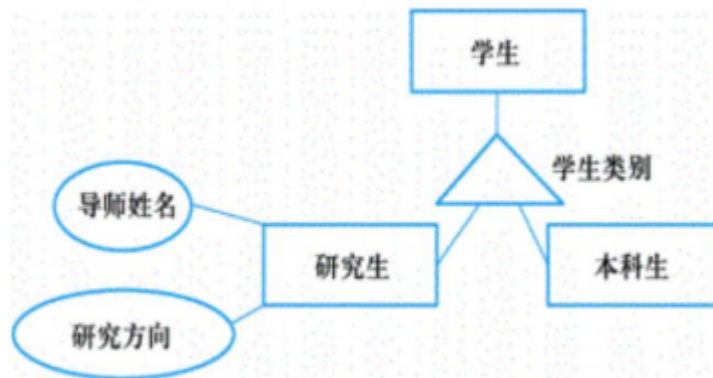
3.3 扩展的ER模型

3.3.1 ISA联系

有的实体型是某个实体型的子类型，这种父类-子类联系称为ISA联系。用 Δ 表示。

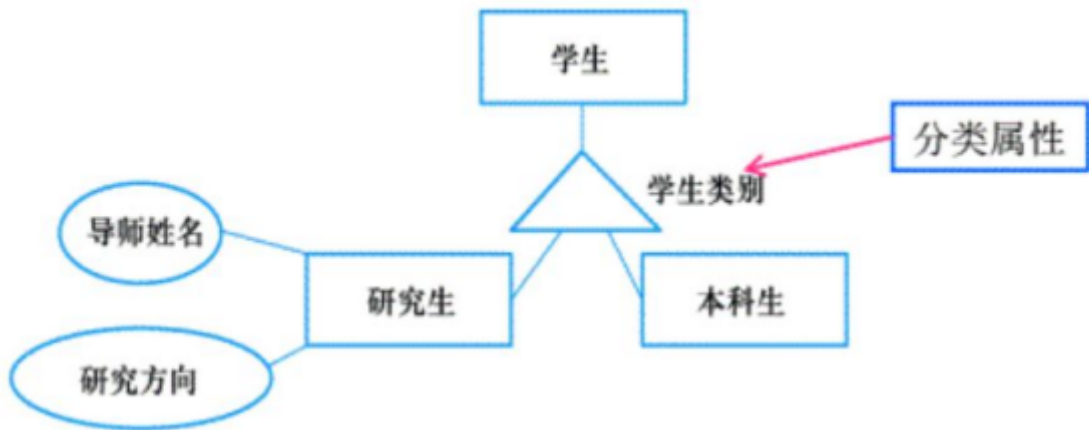
性质：

- 子类继承了父类的所有属性
- 子类也可以有自己的属性



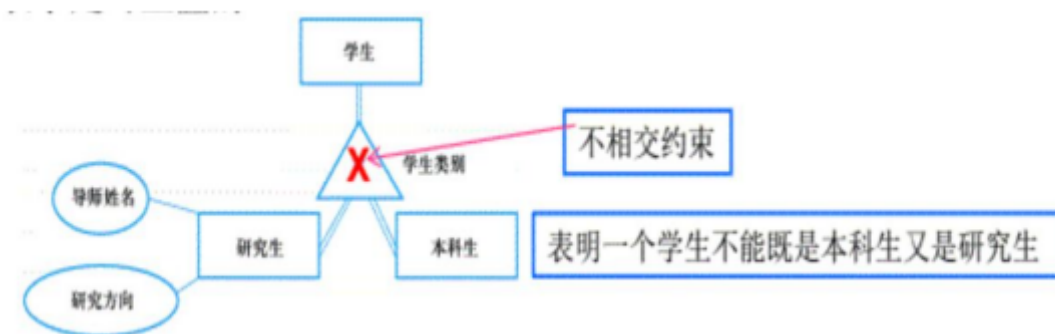
学生的2个子类

分类属性：父实体型的一个属性

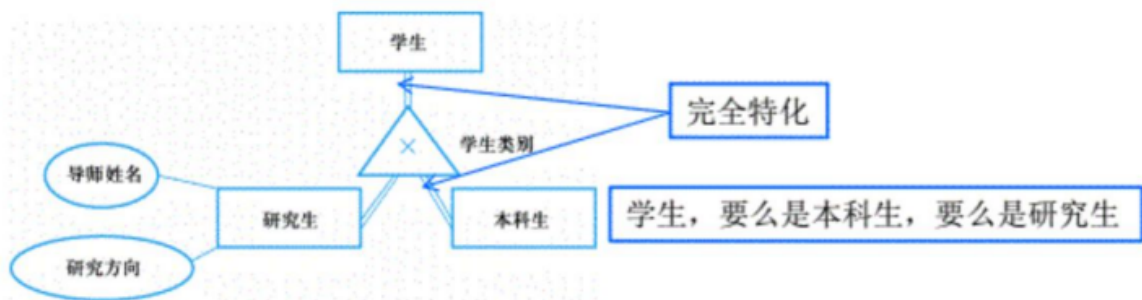


不相交约束：父类中的一个实体不能同时属于多个子类中的实体集。用×表示。

可重叠约束：允许父类中的一个实体能同时属于多个子类中的实体集。



完备性约束：描述父类中的一个实体是否必须是某一个子类中的实体。如果是，称为完全特化，否则称为部分特化。完全特化用双线表示，部分特化用单线表示。



3.3.2 基数约束

说明实体型中的任何一个实体可以在联系中出现的最少次数和最多次数。

是对一对一、一对多、多对多的细化。

约束用一个数对 $min \dots max$ 表示, $0 \leq min \leq max$ 。

- 实体E在联系R中的基数约束 $\text{card}(E, R)$ ，可以用一个数对 $\text{min}.. \text{max}$ 来表示
- 其中：

- 最小次数 min 被称为实体E在联系R中的**最小参与基数**，取值为 0 或 1
- 最大次数 max 被称为实体E在联系R中的**最大参与基数**，取值为 1 或 *
(* 代表‘多’，指大于1的任意数目，也可以用任意一个字母来代表‘多’)

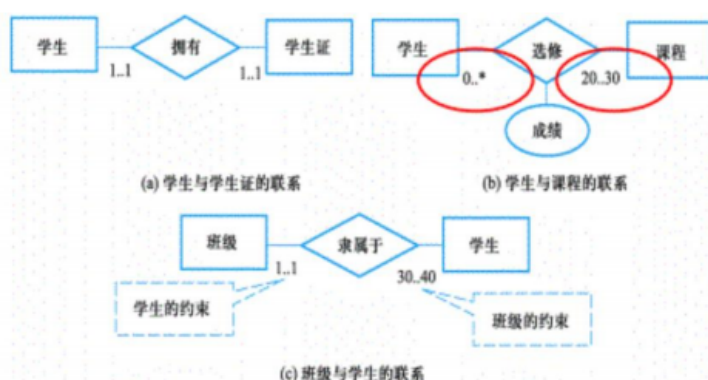
$\text{min} = 1$	要求E中的每一个实体在联系R中至少出现一次
$\text{min} = 0$	允许E中的某些实体在联系R中不出现
$\text{max} = 1$	要求E中的每一个实体在联系R中最多只出现一次
$\text{max} = *$	允许E中的某些实体在联系R中可以出现多次



- $\text{card}(\text{汽车}, \text{拥有}) = 1..*$
- $\text{card}(\text{轮子}, \text{拥有}) = 0..1$



- $\text{card}(\text{贷款}, \text{还贷}) = 0..*$
- $\text{card}(\text{还款}, \text{还贷}) = 1..1$

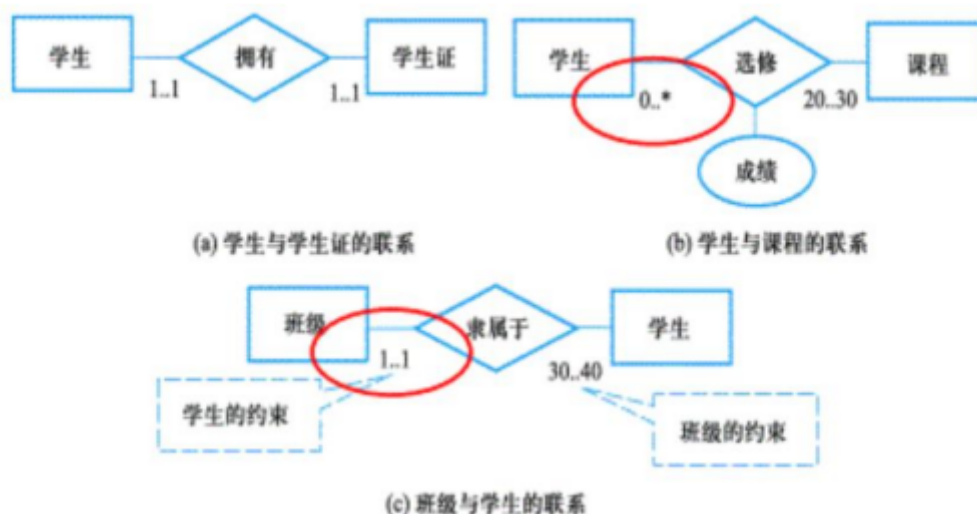


学生实体的基数约束是20..30
表示每个学生必须选修20~30门课程：

课程的一个基数约束是0..*，即
一门课程可以被很多同学选修也
可能还没有同学选修，如新开课。

$\text{min} = 1$ 的约束称为强制参与约束，即每个实体都要参与联系。

$\text{min} = 0$ 的约束称为非强制参与约束。



$\text{max} = 1$ 的约束称为单值参与约束，即被施加基数约束的实体型中的每个实体在联系中至多出现一次。

$\text{max} = *$ 的约束称为多值约束，可以出现多次。



- ‘汽车’与‘拥有’是多值参与约束
- ‘轮子’与‘拥有’是单值参与约束



- ‘贷款’与‘还贷’是多值参与约束
- ‘还款’与‘还贷’是单值参与约束

3.3.3 part_of 联系

描述某个实体型是另外一个实体型的一部分。

- 非独占联系：整体实体如果被破坏，另一部分实体仍然可以独立存在
- 独占联系：整体被破坏，部分实体不能存在

表示方法：

- 用非强制参与联系表示非独占联系
- 用弱实体类型和识别联系来表示独占联系编号



(图7.15)
用非强制参与约束 0..1
表示非独占Part-of联系



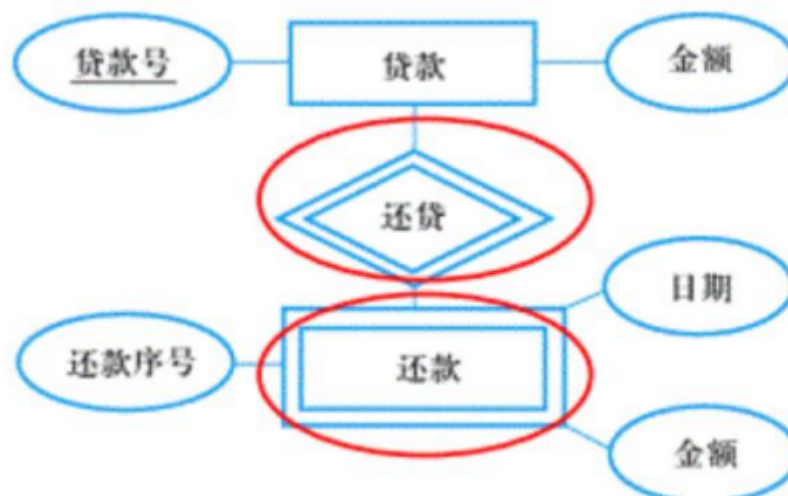
(图7.16)
用双框矩形表示弱实体
用双框菱形表示识别联系

3.3.4 弱实体

如果一个实体型的存在依赖于其他实体型的存在，则这个实体型叫做弱实体型，否则叫做强实体型。

双矩形表示弱实体型，双菱形表示识别联系。

弱实体的表示：双框矩形+从弱实体到联系的有向箭头



3.3.5 属性的划分

根据属性在区分不同实体中的作用：

- 标识符：关键字、码（在ER图中用下划线表示）
- 描述符

根据属性域的定义：

- 单值属性
- 组合属性
- 多值属性（在ER图中用实体型和属性之间的双线段表示）

3.3.6 属性基数

❑ 也可以用一个二元组 (x, y) 来描述一个实体在该属性上取值的数量特征。其中， x 和 y 的含义如下：

$(0, ?)$	x 为0，意味着该属性‘可以取空值’，或者说，允许一个实体在该属性上可以没有确定的值。
$(1, ?)$	x 为1，意味着每一个实体在该属性上都必须有值，‘不能取空值’。
$(?, 1)$	y 为1，意味着每一个实体在该属性上最多只能有一个值，即该属性为单值属性 (single-valued attribute or composite attribute)。
$(?, *)$	y 为* (表示‘多’)，意味着允许一个实体在该属性上的取值是一个集合，即该属性为多值属性 (multi-valued attribute)。

当有了属性基数约束后，就不需要用双线段表示多值属性了。

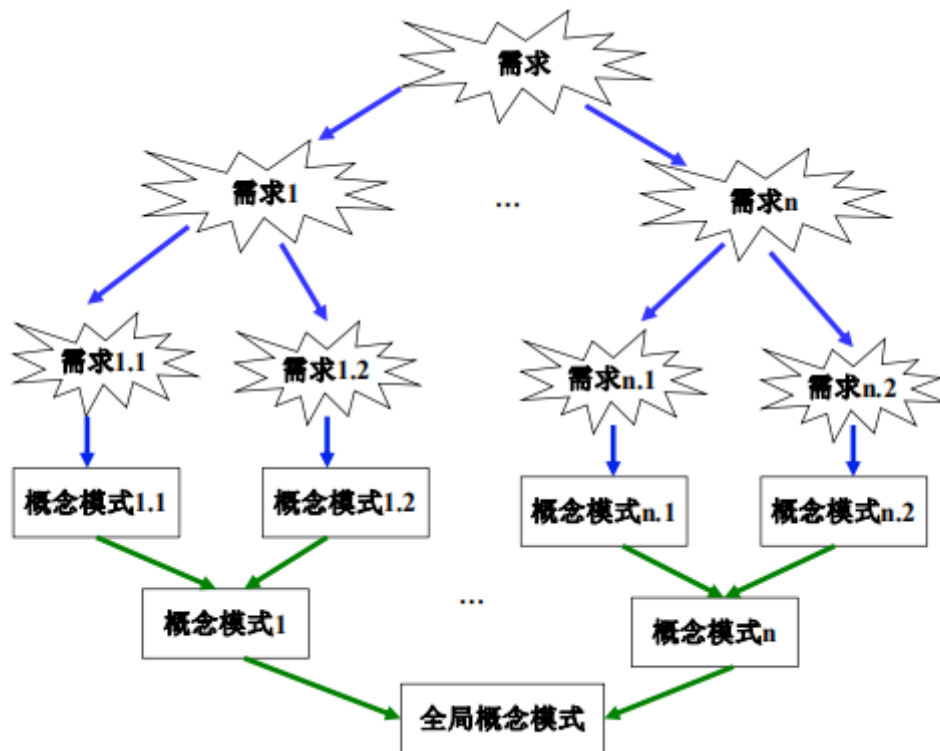
3.4 UML

3.5 概念结构设计

3.5.1 概念结构设计的方法

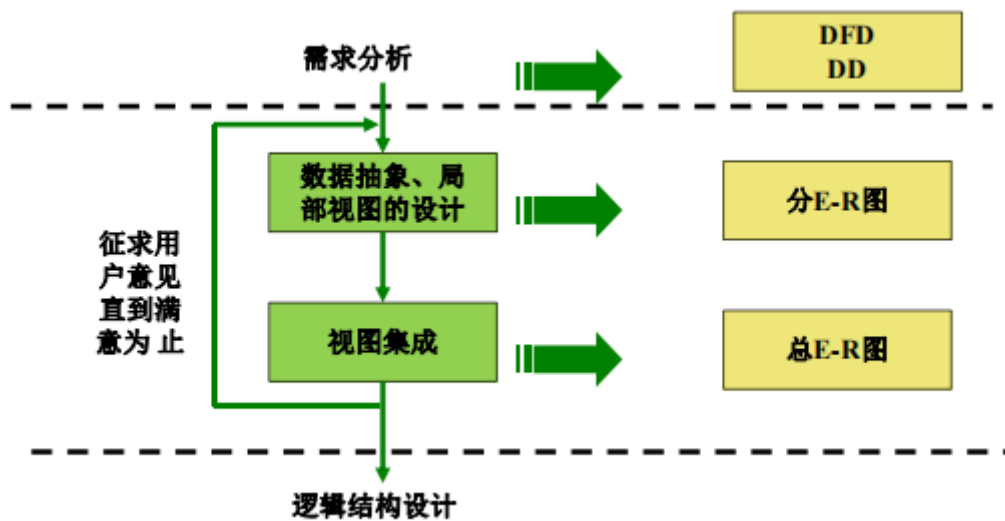
- 自顶向下：首先定义全局概念结构的框架，然后逐步细化
- 自底向上：首先定义各局部应用的概念结构，然后将它们集成起来
- 逐步扩张：首先定义最重要的核心概念结构，然后向外扩充，逐步生成其他概念结构，直至总体概念结构（类似软件工程中的螺旋模型）

混合策略：将自顶向下和自底向上相结合，用自顶向下策略设计一个全局概念结构的框架，以它为骨架集成自底向上策略中设计的各局部概念结构。自顶向下进行需求分析，自底向上设计概念结构。



自底向上概念结构设计的步骤：

- 抽象数据设计局部视图
- 集成局部视图得到全局概念结构



3.5.2 实体与属性的划分原则

- 属性必须不可分
- 属性不能与其他实体之间有联系

3.5.3 ER图的集成

步骤：

- 合并：解决各分ER图之间的冲突，将分ER图合并起来生成初步ER图
- 修改和重构：消除不必要的冗余，生成基本ER图

3.5.3.1 合并

各个子系统的ER图之间存在的**不一致**的地方就是冲突。

分为：

- 属性冲突：
 - 属性**域**冲突：属性值的类型、取值范围或取值集合不同
 - 属性取值**单位**冲突
- 命名冲突：
 - 同名异义：不同意义的对象在不同的局部应用中具有相同的名字
 - 异名同义：同一意义的对象在不同的局部应用中具有不同的名字
- 结构冲突：
 - 同一对象在不同应用中具有不同的抽象（这个抽象指的是属性、实体、联系这些）。解决方法：变换为同一抽象。
 - 同一实体在不同子系统的ER图所包含的属性个数和属性排列次序不完全相同。解决方法：使该实体的属性取各子系统的ER图中属性的并集，再适当调整属性的次序。
 - 实体间的联系在不同的ER图中为不同的类型。解决方法：根据应用的语义对实体联系的类型进行综合或调整。

3.5.3.2 修改和重构

消除不必要的冗余。主要使用分析方法，以DD和DFD为依据，根据DD中关于数据项之间逻辑关系的说明来消除冗余。

- 并不是所有的冗余数据和联系都需要消除，有时为了提高效率不得不以冗余信息为代价

❑ 如图7.26中， $Q_3 = Q_1 \times Q_2$ ， $Q_4 = \sum Q_5$ 。所以 Q_3 和 Q_4 是冗余数据，可以消去。

❑ 并且，由于 Q_3 被消去，产品与材料间 $m:n$ 的冗余联系也应被消去。

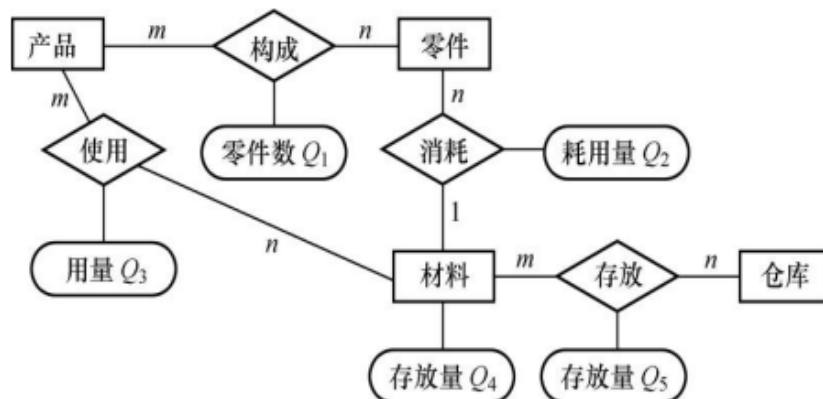
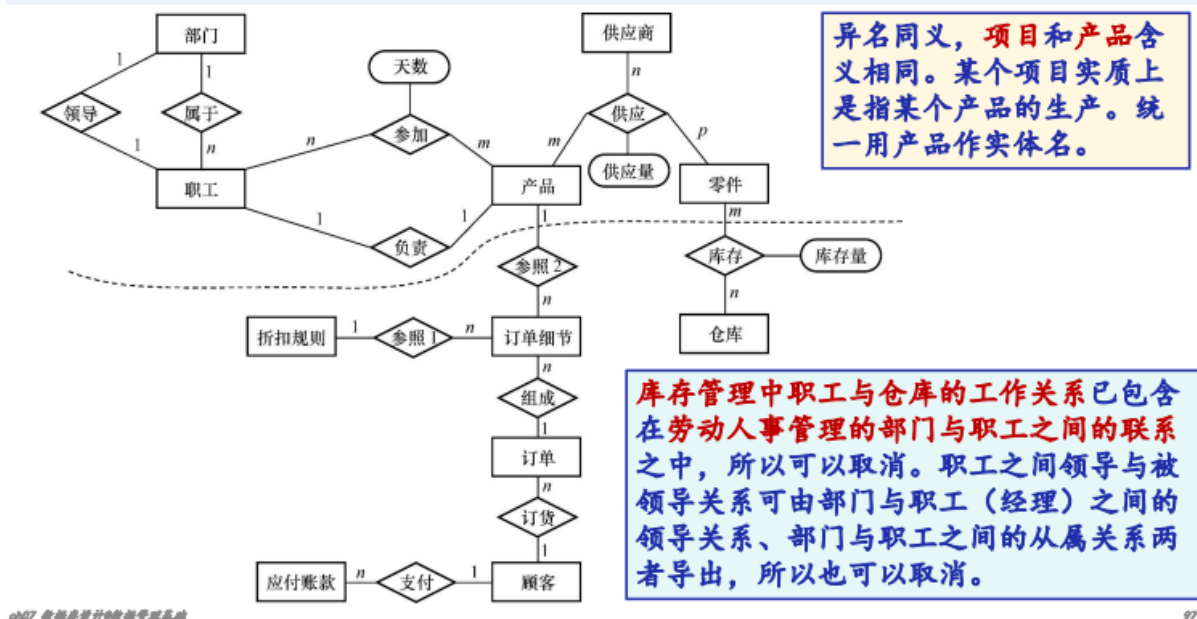


图7.26 消除冗余

集成案例：

图7.28 某工厂管理信息系统的基本E-R图（E-R图集成案例）



4.逻辑结构设计

任务：把概念结构设计阶段设计好的基本ER图转换为与选用数据库管理系统产品所支持的数据模型相符合的逻辑结构。

转换内容：将ER图转换为关系模型

- 将实体型、实体的属性和实体型之间的联系转化为关系模式
- 模型优化
- 设计用户子模式

4.1 转换原则

4.1.1 实体型

关系的属性：实体的属性

关系的码：实体的码

4.1.2 1:1联系

□ 一个1:1联系可以转换为一个独立的关系模式，也可以与任意一端对应的关系模式合并。

□ 转换为一个独立的关系模式

- 关系的属性：与该联系相连的各实体的码以及联系本身的属性
- 关系的候选码：每个实体的码均是该关系的候选码

□ 与某一端实体对应的关系模式合并

- 合并后关系的属性：加入对应关系的码和联系本身的属性
- 合并后关系的码：不变

4.1.3 1:n联系

□ 一个1:n联系可以转换为一个独立的关系模式，也可以与n端对应的关系模式合并。

□ 转换为一个独立的关系模式

- 关系的属性：与该联系相连的各实体的码以及联系本身的属性
- 关系的码：n端实体的码

□ 与n端对应的关系模式合并

- 合并后关系的属性：在n端关系中加入1端关系的码和联系本身的属性
- 合并后关系的码：不变
- 可以减少系统中的关系个数

4.1.4 m:n联系

□ 一个m:n联系转换为一个关系模式

- 关系的属性：与该联系相连的各实体的码以及联系本身的属性
- 关系的码：各实体码的组合

□ [例] “选修”联系是一个m:n联系，可以将它转换为如下关系模式，其中学号与课程号为关系的组合码：

- 选修 (学号, 课程号, 成绩)

4.1.5 多元联系

三个或三个以上实体间的一个多元联系转换成一个关系模式。

关系的属性：与该多元联系相连的各实体的码以及联系本身的属性。

关系的码：各实体码的组合。

4.1.6 关系模式的合并

具有相同的码的关系模式可以合并。

目的：减少系统中的关系个数。

方法：

- 将其中一个关系模式的全部属性加入到另一个关系模式中
- 然后去掉其中的同义属性（可能同名也可能不同名）
- 适当调整属性的次序

4.2 模型优化

关系数据模型的优化通常以规范化理论为指导。

4.2.1 关系规范化设计

确定数据依赖+规范化设计（确定各关系模式属于第几范式，考察部分函数依赖、传递函数依赖、多值依赖等）。

不同关系之间冗余函数依赖的检查。对各个关系模式之间的数据依赖进行极小化处理，消除冗余的联系。

并不是规范化程度越高的关系越优！

4.2.2 关系模式的分解

- 水平分解：把（基本）关系的元组集合划分为若干子集合，定义每个子集合为一个子关系。

如何分解：对符合“80/20原则”的，把经常被使用的数据（约20%）水平分解出来，形成一个子关系。

- 垂直分解：把关系模式R的属性集划分为若干子集合，形成若干子关系模式。

原则：经常在一起使用的属性分解出来形成一个子关系模式。

优缺点：提高某些事务效率；可能使得一些事务不得不执行连接操作，降低了效率。

分解方法：

- 简单情况：直观分解
- 复杂情况：适用模式分解算法（垂直分解必须不损失关系模式的语义，保持无损连接性和保持函数依赖）

4.3 设计用户子模式

- 定义数据库模式主要是从系统的时间效率、空间效率、易维护等角度出发
- 定义用户子模式时更注意用户的习惯与方便：
 - 适用更符合用户习惯的别名
 - 针对不同级别的用户定义不同的视图，保证系统的安全性

- 简化用户对系统的使用

DBMS适应性转换：一般的数据模型还需要向特定的DBMS规定的模型进行转换。转换的主要依据是所选用的DBMS的功能和限制。

5.物理结构设计

5.1 定义

数据库在**物理设备上的存储结构与存取方法**称为数据库的物理结构。

为一个给定的逻辑数据模型选取一个最适合应用要求的物理结构的过程就是数据库的物理设计。

步骤：

- 确定数据库的物理结构
- 对物理结构进行评价，评价重点是时间和空间效率
- 若评价结果满足原设计要求，则可以进入到物理实施阶段，否则需要重新设计或修改物理结构

5.2 数据库物理设计的内容和方法

数据库物理设计的内容和方法

□ 设计物理数据库结构的准备工作

- 充分了解应用环境，详细分析要运行的事务，以获得选择物理数据库设计所需参数。
- 充分了解所用关系型数据库管理系统的内部特征，特别是系统提供的存取方法和存储结构。

□ 关系数据库物理设计的内容

- 为关系模式选择存取方法
- 设计关系、索引等数据库文件的物理存储结构

□ 物理数据库设计所需参数

- 数据库查询事务
 - 查询的关系
 - 查询条件所涉及的属性
 - 连接条件所涉及的属性
 - 查询的投影属性
- 数据更新事务
 - 被更新的关系
 - 每个关系上的更新操作条件所涉及的属性
 - 修改操作要改变的属性值
- 每个事务在各关系上运行的频率和性能要求

5.3 关系模式存取方法选择

常用的存取方法：

- B+树索引存取
- Hash索引存取
- 聚簇存取

5.3.1 B+树索引存取

□ 选择索引存取方法的主要内容：根据应用要求确定

- 对哪些属性列建立索引
- 对哪些属性列建立组合索引
- 对哪些索引要设计为唯一索引

□ 选择索引存取方法的一般规则

- 如果一个（或一组）属性经常在查询条件中出现，则考虑在这个（或这组）属性上建立索引（或组合索引）
- 如果一个属性经常作为最大值和最小值等聚集函数的参数，则考虑在这个属性上建立索引
- 如果一个（或一组）属性经常在连接操作的连接条件中出现，则考虑在这个（或这组）属性上建立索引

□ 关系上定义的索引数过多会带来较多的额外开销（维护、查找索引的开销）

5.3.2 Hash存取

选择Hash存取方法的规则：

如果一个关系的属性主要出现在等值连接条件中或主要出现在等值比较选择条件中，而且满足下列两个条件之一：

- 该关系的大小可预知，而且不变；
- 该关系的大小动态改变，但所选用的数据库管理系统提供了动态Hash存取方法。

5.3.3 聚簇索引

聚簇：为了提高某个属性（或属性组）的查询速度，把这个或这些属性（称为聚簇码）上**具有相同值的元组集中存放在连续的物理块**中称为聚簇。

适用范围：

- 既适用于单个关系独立聚簇
- 也适用于经常进行连接操作的多个关系

把多个连接的元组按连接属性值聚集存放；从而实现多个关系的“预连接”，提高连接操作的效率

选择聚簇存储方法，即确定需要建立多少个聚簇，每个聚簇中包含哪些关系。

一个数据库可以建立多个聚簇，一个关系只能加入一个聚簇。

设计候选聚簇：

- 常在一起进行连接操作的关系可以建立组合聚簇
- 如果一个关系的一组属性经常出现在相等比较条件中，则该单个关系可以建立聚簇
- 如果一个关系的一个（或一组）属性上的值重复率很高，则此单个关系可以建立聚簇

检查候选聚簇中的关系，取消其中不必要的关系：

- 从聚簇中删除经常进行全表扫描的关系
- 从聚簇中删除更新操作远多于连接操作的关系
- 从聚簇中删除重复出现的关系

当一个关系同时加入多个聚簇时，必须从这多个聚簇方案（包括不建立聚簇）中选择一个较优的，即在这个聚簇上运行各种事务的总代价最小

局限性：

- 聚簇只能提高某些特定应用的性能
- 建立与维护聚簇的开销相当大

5.4 确定数据库的存储结构

主要指确定数据的存放位置和存储结构，包括：确定关系、索引、聚簇、日志、备份等的存储安排和存储结构，确定系统配置等。

影响因素：

- 硬件环境
- 应用需求：存取时间，存储空间利用率，维护代价

基本原则：

- 将易变部分与稳定部分分开
- 经常存取部分和存取频率较低部分分开

确定系统配置：

❑ 数据库管理系统一般都提供了一些存储分配参数

- | | |
|------------------------|-----------|
| ➤ 同时使用数据库的用户数 | ➤ 存储分配参数 |
| ➤ 同时打开的数据库对象数 | ➤ 物理块的大小 |
| ➤ 内存分配参数 | ➤ 物理块装填因子 |
| ➤ 缓冲区分配参数（使用的缓冲区长度、个数） | ➤ 时间片大小 |
| ➤ | ➤ 数据库的大小 |
| | ➤ 锁的数目 |
| | ➤ |

❑ 系统都为这些变量赋予了合理的缺省值。

❑ 在进行物理设计时需要根据应用环境确定这些参数值，以使系统性能最优。

❑ 在物理设计时对系统配置变量的调整只是初步的，要根据系统实际运行情况做进一步的调整，以切实改进系统性能。

5.5 评价物理结构

评价标准：

- 存储空间
- 存取时间
- 维护代价

6.数据库的实施和维护

6.1 数据的载入和应用程序的调试

6.1.1 数据的载入

数据装载方法：

- 人工方法
- 计算机辅助数据入库

6.1.2 应用程序的调试

数据库应用程序的设计应该与数据设计并行进行。

在组织数据入库的同时还要调试应用程序。

6.2 数据库试运行

主要工作：

- 功能测试：实际运行应用程序，执行对数据库的各种操作，测试程序的各种功能
- 性能测试：测量系统的性能指标，分析是否符合设计目标

试运行的注意事项：

① 数据的分期入库

- 重新设计物理结构甚至逻辑结构，会导致数据重新入库
- 由于数据入库工作量实在太大，所以可以采用分期输入数据的方法
 - 先输入小批量数据供先期联合调试使用
 - 待试运行基本合格后再输入大批量数据
 - 逐步增加数据量，逐步完成运行评价

② 数据库的转储和恢复

- 在数据库试运行阶段，系统还不稳定，硬、软件故障随时都可能发生
- 系统的操作人员对新系统还不熟悉，误操作也不可避免
- 因此必须做好数据库的转储和恢复工作，尽量减少对数据库的破坏

6.3 数据库的运行和维护

6.3.1 转储与恢复

数据库管理员要针对不同的应用要求制定不同的转储计划，定期对数据库和日志文件进行备份。

一旦发生介质故障，即利用数据库备份及日志文件备份，尽快将数据库恢复到某种一致性状态。

6.3.2 安全性和完整性控制

初始定义：

- 数据库管理员根据用户的实际需要授予不同的操作权限
- 根据应用环境定义不同的完整性约束条件

修改定义：

- 当应用环境发生变化，对安全性的要求也会发生变化，数据库管理员需要根据实际情况修改原有的安全性控制
- 修正数据库的完整性约束条件

6.3.3 数据库性能的监督、分析和改造

- 利用监测工具获取系统运行过程中一系列性能参数的值
- 通过仔细分析这些数据，判断当前系统是否处于最佳运行状态
- 如果不是，则需要通过调整某些参数来进一步改进数据库性能

6.3.4 数据库的重组织和重构造

重组织：

□ 为什么要重组织数据库

- 数据库运行一段时间后，由于记录的不断增、删、改，会使数据库的物理存储变坏，从而降低数据库存储空间的利用率和数据的存取效率，使数据库的性能下降。

□ 重组织的形式

- 全部重组织
- 部分重组织
 - 只对频繁增、删的表进行重组织

□ 重组织的目标

- 提高系统性能

□ 重组织的工作

- 按原设计要求
 - 重新安排存储位置
 - 回收垃圾
 - 减少指针链
- 数据库的重组织不会改变原设计的数据逻辑结构和物理结构

- 数据库管理系统一般都提供了供重组织数据库使用的实用程序，帮助数据库管理员重新组织数据库。

重构造：

□ 为什么要进行数据库的重构造

- 数据库应用环境发生变化，会导致实体及实体间的联系也发生相应的变化，使原有的数据库设计不能很好地满足新的需求
 - 增加新的应用或新的实体
 - 取消某些已有应用
 - 改变某些已有应用

□ 重构造数据库的程度是有限的

- 若应用变化太大，已无法通过重构数据库来满足新的需求，或重构数据库的代价太大，则表明现有数据库应用系统的生命周期已经结束，应该重新设计新的数据库应用系统了。

□ 重构造的主要工作

- 根据新环境调整数据库的模式和内模式
 - 增加或删除某些数据项
 - 改变数据项的类型
 - 增加或删除某个表
 - 改变数据库的容量
 - 增加或删除某些索引